

CSE 715: Computer Graphics

Z-Buffer Algorithm — Chapter 10, Hidden Surfaces (§10.1–10.2)

Complete Model Answers (2020–2024)

University of Chittagong — 7th Semester B.Sc. (Engg.) Examination

Source: Schaum's Outline of Computer Graphics (Xiang & Plastock), §10.1–10.2 — notation follows the book exactly ($Z_{\text{buf}}(x, y)$, Z_{back})

Contents

1	Theoretical Framework	2
1.1	Depth Comparison (§10.1)	2
1.2	The Z-Buffer Algorithm (§10.2)	2
1.3	Painter's Algorithm vs. Z-Buffer (for the differentiate-style question)	3
2	Examination 2022 — Q6(c) & 2021 — Q6(d): Full Trace	3
3	Examination 2023 — Q7(d): Painter's vs. Z-Buffer [2 marks]	4
4	Examination 2024 — Q7(d): Illustrate + Pros/Cons [2 marks]	5
5	Quick Summary	5

Theoretical Framework

Yield note: re-verified against all 5 raw scanned papers 2026-07-06. The full "max objects + 2×2 pixel trace" question is only **2021 (Q6d) and 2022 (Q6c)**. 2023 (Q7d) asks a lighter "differentiate Painter's vs. Z-buffer" instead, and 2024 (Q7d) asks "illustrate + advantages/disadvantages." **2020 has no Z-buffer question at all** — that year's Ch10 content is only the separate "point-obscures-point via viewpoint" topic (Q8c), tracked independently.

Depth Comparison (§10.1)

Given two points $P_1(x_1, y_1, z_1)$ and $P_2(x_2, y_2, z_2)$ on the *same projection line* (i.e. the same pixel), a depth comparison tells us which is in front. For an orthographic parallel projection onto the xy plane with the viewing direction along $+z$ (viewer at $-z$, looking toward $+z$): **if $z_1 < z_2$, then P_1 obscures P_2** (smaller z = closer to the viewer). All hidden-surface algorithms in this chapter assume coordinates already in **normalized viewing coordinates**, with any perspective view first converted to an equivalent parallel one via the perspective-to-parallel transform T_p , so depth comparison always reduces to this simple z_1 vs. z_2 test.

The Z-Buffer Algorithm (§10.2)

An **image-space** method. Maintain a Z-buffer $Z_{\text{buf}}(x, y)$ — the depth of the closest point seen so far at each pixel (x, y) .

1. **Initialize:** set every pixel to the background color; set $Z_{\text{buf}}(x, y) = Z_{\text{back}}$ (the depth of the back clipping plane) for every pixel.
2. **Scan-convert each (already-projected) polygon P in the scene.** For every pixel (x, y) inside P : compute $Z(x, y)$, the depth of P at that pixel.
 - If $Z(x, y) < Z_{\text{buf}}(x, y)$: set $Z_{\text{buf}}(x, y) = Z(x, y)$ and set the pixel's color to P 's color at (x, y) (this polygon is closer than anything seen so far at this pixel).
 - Otherwise: leave the pixel and Z_{buf} unchanged (something already drawn is closer).

Maximum number of objects representable: unlimited / arbitrarily large.

The algorithm requires Z-buffer memory proportional only to the number of *pixels* on screen — it processes polygons *one at a time* and never needs to store all the scene's objects simultaneously, so the total object count in a scene is not bounded by the algorithm itself. **Advantages:** simple to implement; handles an arbitrary/unlimited number of objects; no need to sort or resolve polygon-polygon depth ordering ambiguities (unlike Painter's algorithm); processes polygons independently, in any order. **Disadvantages:** needs Z-buffer memory proportional to screen resolution

(one depth value per pixel, in addition to the frame buffer); can suffer from depth-precision/aliasing artifacts; doesn't natively handle transparency (only the single nearest opaque surface per pixel is kept); no inherent anti-aliasing at silhouette edges.

Painter's Algorithm vs. Z-Buffer (for the differentiate-style question)

	Painter's Algorithm	Z-Buffer
Method	Depth-sort: paint polygons farthest-to-nearest by priority order	Per-pixel depth compare; keep closest seen so far
Ordering effort	Must sort by $z_{\max}$, then resolve overlap ambiguities (Tests 0–5); may need to split polygons	No sorting needed — polygons processed in any order
Memory	No extra per-pixel depth storage, but needs polygon list & ambiguity-resolution structures	Needs one Z-buffer entry per pixel
Ambiguity handling	Cyclic/overlapping polygons can force polygon splitting	None — simple numeric compare always resolves it
Object count limit	Sorting cost grows with polygon count	Effectively unlimited (memory is per-pixel, not per-object)

Examination 2022 — Q6(c) & 2021 — Q6(d): Full Trace

2022 Q6(c) [4.5 marks]: What is the maximum number of objects that can be presented by using the Z-buffer algorithm? Using a 2×2 pixel display, show how the Z-buffer algorithm would determine the color of each pixel for the given objects A and B in the figure (a box scene with a back clipping plane, objects A and B shown at different depths, pixel coordinates 0/1/2 on both axes).

2021 Q6(d) [4.75 marks]: Identical question structure with a different A/B depth arrangement in the figure.

Part 1 — Maximum Number of Objects

Unlimited / arbitrarily large — see §1.2. The Z-buffer's memory requirement depends only on screen resolution (one depth value per pixel), not on how many

objects are in the scene, since objects are scan-converted and compared one at a time.

Part 2 — 2×2 Pixel Trace Method

The exact depth values are read off each year's own diagram (a 3D box showing objects A and B at different distances behind the display screen, with pixel coordinates 0, 1, 2 on both axes of a 2×2 display). Apply the method below directly using your paper's specific A/B positions and depths — the mechanics never change, only which of A/B is closer at each of the 4 pixels.

Step 1 — Initialize: for all 4 pixels (0, 0), (1, 0), (0, 1), (1, 1), set color = background, $Z_{\text{buf}} = Z_{\text{back}}$ (the depth of the rear clipping plane, the largest possible depth).

Step 2 — Scan-convert object A first (arbitrary order): for every pixel that A's projection covers, read A's depth $Z_A(x, y)$ at that pixel. Since $Z_A < Z_{\text{back}}$ everywhere A is drawn, update $Z_{\text{buf}}(x, y) = Z_A(x, y)$ and set that pixel's color to A's color.

Step 3 — Scan-convert object B: for every pixel B covers, read $Z_B(x, y)$ and compare against the current $Z_{\text{buf}}(x, y)$:

- At any pixel covered by *both* A and B: whichever of Z_A, Z_B is **smaller** (nearer the viewer) wins — that object's color is the final pixel color.
- At a pixel covered only by B (A didn't reach it): $Z_B < Z_{\text{buf}} = Z_{\text{back}}$, so B's color wins there.
- At a pixel covered by neither: the color stays background.

Worked illustrative example (representative numbers, method identical to either year's actual figure): suppose A occupies pixels (0, 0) and (1, 0) at depth $z = 3$; B occupies pixels (1, 0) and (1, 1) at depth $z = 2$; pixel (0, 1) is covered by neither.

Pixel	Covered by	Depths seen	Winner (min z)	Final color
(0, 0)	A only	$z_A = 3$	A	color of A
(1, 0)	A and B	$z_A = 3, z_B = 2$	B ($2 < 3$)	color of B
(0, 1)	neither	—	—	background
(1, 1)	B only	$z_B = 2$	B	color of B

Substitute your own paper's A/B pixel coverage and depth values into this same table structure — the answer is always "smallest z wins at each pixel; background where neither object is drawn."

Examination 2023 — Q7(d): Painter's vs. Z-Buffer [2 marks]

Question: Differentiate between the Painter’s algorithm and the Z-buffer algorithm.

See the comparison table in §1.3. Core distinction to lead with: Painter’s algorithm is a **depth-sort / priority** method that paints whole polygons in far-to-near order and must resolve sorting ambiguities (possibly splitting polygons); the Z-buffer is a **per-pixel numeric-comparison** method requiring no sorting at all, at the cost of needing a full-screen depth buffer in memory.

Examination 2024 — Q7(d): Illustrate + Pros/Cons [2 marks]

Question: Illustrate the Z-buffer algorithm with necessary figures. What are its advantages and disadvantages?

Illustration: draw the initialize/scan-convert/compare-and-update flow from §1.2 (a simple flowchart: Initialize $Z_{\text{buf}} = Z_{\text{back}}$ for all pixels $\rightarrow$ for each polygon, for each covered pixel, compute $Z(x, y) \rightarrow$ if $Z(x, y) < Z_{\text{buf}}(x, y)$ update buffer + pixel color $\rightarrow$ repeat for all polygons). **Advantages/disadvantages:** see §1.2 — lead with “handles unlimited objects, no sorting needed” as the main advantage and “needs full-screen depth memory, no native transparency/anti-aliasing” as the main disadvantage.

Quick Summary

Pattern	Years	Method
Max objects + 2×2 pixel trace (full numeric)	2021, 2022	Unlimited objects (memory is per-pixel); trace: smallest z wins at each pixel, background where nothing drawn
Differentiate Painter’s vs. Z-buffer	2023	Depth-sort+ambiguity-resolution vs. per-pixel numeric compare
Illustrate + advantages/disadvantages	2024	Flowchart of init/compare/update; unlimited objects vs. full-screen memory cost

Exam-day takeaway: whatever form it takes, the answer always centers on the same fact — **Z-buffer compares depths per pixel, keeps the smallest, and needs one depth value per pixel rather than a bound on object count.** Memorize

that one sentence and you can answer any of the four question shapes above.